FAST – NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Department of Computer Science

Database Management Systems — Spring 2026

PROJECT REPORT

Intelligent Inventory Management System

A Modern, Data-Driven Inventory & E-Commerce Platform

Real-time Inventory Tracking	AI-Powered Recommendations	Dual-Module Architecture

PROJECT TEAM

Tuheed Khan	Anas	Waqar Nadeem
24K-2547	24K-2560	24K-2524
Frontend & Debugging	Backend, DB & Testing	Integration & Debugging

1. Introduction

1.1 Aim & Motivation

Traditional inventory management systems rely on manual processes and file-based storage, leading to inefficiencies, data inconsistencies, and missed opportunities for data-driven decision-making. Small to medium businesses frequently suffer from stockouts, order errors, and lack visibility into customer behavior — problems that directly translate to lost revenue and poor customer satisfaction.

The Intelligent Inventory Management System (IIMS) was conceived to close this gap. By replacing flat-file records with a fully normalized relational database and wrapping it with a modern web application, IIMS transforms inventory management from a back-office burden into a strategic business asset. The system empowers administrators with real-time stock visibility and demand forecasting while offering customers a personalized, responsive e-commerce experience.

1.2 Project Context

This project was developed as the semester capstone for the Database Management Systems (DBMS) course at FAST-NUCES, Spring 2026. It demonstrates practical application of database design, normalization, query optimization, transaction management, and web integration using an industry-relevant use case.

2. Background — Research & Project Selection

During the project selection phase, the team evaluated three candidate ideas: a Hospital Management System, a University ERP module, and an Inventory & E-Commerce Platform. The inventory platform was selected for the following reasons:

- Richness of relational schema — seven interconnected entities spanning users, products, categories, orders, and audit trails.
- Dual-user architecture — naturally exercises role-based access control and separate data views.
- Analytical depth — supports market basket analysis, demand forecasting, and customer segmentation queries, covering advanced SQL concepts.
- Real-world relevance — directly applicable to the local retail and e-commerce landscape in Pakistan.

The team surveyed open-source inventory tools (ERPNext, Odoo, Snipe-IT) to understand existing approaches and identify what a purpose-built academic implementation should prioritise: conceptual clarity, normalization correctness, and demonstrable SQL complexity over deployment scale.

3. Project Specification

3.1 Functional Requirements

#	Module	Requirement
FR-01	Admin	CRUD operations on Products (name, price, stock, category)
FR-02	Admin	Real-time stock level monitoring with low-stock alerts
FR-03	Admin	View and update Order status (Pending → Processing → Shipped → Delivered)
FR-04	Admin	Customer account management and purchase history review
FR-05	Admin	Sales analytics: revenue trends, top products, demand forecasting
FR-06	Admin	Audit log of all product actions (Admin_Products associative entity)
FR-07	Customer	Self-registration, login, and profile management
FR-08	Customer	Browse and search products by category
FR-09	Customer	Add to cart, checkout, and order placement
FR-10	Customer	View personalised product recommendations
FR-11	Customer	Order tracking and return/refund requests
FR-12	Both	Automated transactional email notifications for account lifecycle events

3.2 Non-Functional Requirements

- Performance: Query response time under 500 ms for standard CRUD operations on up to 10,000 product records.
- Security: Passwords hashed with bcrypt; JWT tokens for session management; SQL injection prevention via prepared statements.
- Scalability: Schema designed to support horizontal table partitioning for future growth.
- Data Integrity: Full referential integrity enforced via foreign key constraints; ACID-compliant transactions for order processing.
- Availability: RESTful API design enables future multi-client support (mobile, third-party integrations).

4. Entity-Relationship Diagram (ERD)

4.1 ERD Overview

The ERD below was designed by the team to capture all entities, attributes, and relationships in the IIMS database. It follows standard Chen notation: rectangles for entities, ellipses for attributes, diamonds for relationships, and double-bordered shapes for associative (weak) entities.

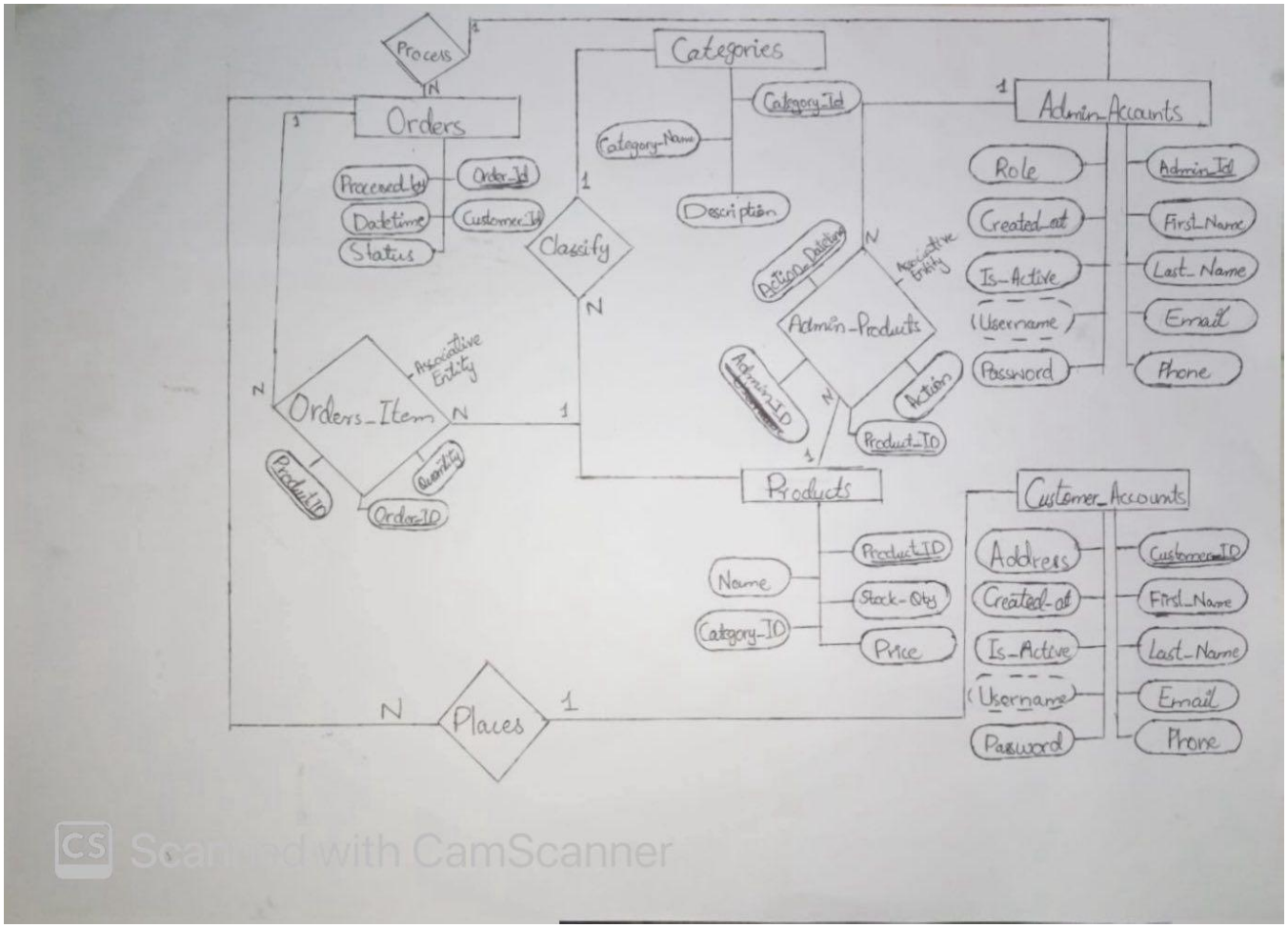


Figure 1 — IIMS Entity-Relationship Diagram (hand-drawn)

4.2 Entity Descriptions

Entity	Type	Key Attribute(s)	Description
Customer_Accounts	Strong	CustomerID	Registered customers with login credentials and contact details.
Admin_Accounts	Strong	Admin_ID	Staff/admin users with role-based permissions.
Products	Strong	Product_ID	Catalogue items with name, price, stock quantity, and category link.
Categories	Strong	Category_ID	Product classification with name and description.

Orders	Strong	Order_ID	Customer order header with status, datetime, and customer reference.
Orders_Item	Associative	Order_ID + Product_ID	Line items linking Orders to Products with quantity (many-to-many resolver).
Admin_Products	Associative	Admin_ID + Product_ID	Audit table recording which admin performed which action on which product.

4.3 Relationships

Relationship	Entities Involved	Cardinality	Notes
Places	Customer_Accounts ↔ Orders	1 : N	One customer places many orders.
Process In	Orders ↔ Admin_Accounts	N : 1	Many orders processed by one admin.
Classify	Orders ↔ Categories	N : N	Orders span multiple categories (via Products).
Orders_Item	Orders ↔ Products	N : N	Resolved by associative entity with Quantity attribute.
Associative Entity	Admin_Accounts ↔ Products	N : N	Admin_Products records action type per product.

4.4 Nomalization

UNF — Unnormalized Form

> *All data stored in one table. Contains a repeating group of product columns (one row per order, with N products embedded). No primary key is defined.*

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
ORDER_MASTER No PK defined	*Repeating group:* ~~order_id, order_datetime, order_status, customer_username, customer_password, customer_first_name, customer_last_name, customer_email, customer_phone, customer_address, admin_username, admin_password, admin_first_name, admin_last_name, admin_email, admin_phone, admin_role, { product_id, product_name, product_price, product_stock, qty, category_id, category_name, category_description } × N~~

1NF — First Normal Form

> *Rule: All values must be atomic (indivisible), no repeating groups, and every row must be uniquely identifiable by a primary key.*

> Action: Eliminated the repeating product group by expanding each product into its own row. Composite PK defined as (order_id, product_id). All values are now atomic.

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
ORDER_ALL	**order_id, product_id, order_datetime,** order_status, customer_username, customer_password, customer_first_name,

PK: (order_id, product_id, order_datetime)	customer_last_name, customer_email, customer_phone, customer_address, admin_username, admin_password, admin_first_name, admin_last_name, admin_email, admin_phone, admin_role, product_name, product_price, product_stock, qty, category_id, category_name, category_description

2NF — Second Normal Form

Rule: Must be in 1NF. Every non-key attribute must depend on the whole primary key — not just part of it (no partial dependencies).

Action: PK is (order_id, product_id). Found two partial dependencies: — Depends only on order_id: all order, customer, and admin columns — Depends on product_id: all product and category columns — Depends on both: qty (remains in the junction table) Split into three separate tables.

ORDERS — partial dependency on order_id only

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
ORDERS PK: order_id	*Stays:* **order_id**, order_datetime, order_status *Transitive (removed in 3NF):* ~~customer_username, customer_password, customer_first_name, customer_last_name, customer_email, customer_phone, customer_address, admin_username, admin_password, admin_first_name, admin_last_name, admin_email, admin_phone, admin_role~~

PRODUCTS — partial dependency on product_id only

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
PRODUCTS PK: product_id	*Stays:* **product_id**, product_name, product_price, product_stock *Transitive (removed in 3NF):* ~~category_id, category_name, category_description~~

ORDER_ITEMS — depends on full composite key

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
ORDER_ITEMS PK: (order_id, product_id)	**order_id**, **product_id**, qty

3NF — Third Normal Form

Rule: Must be in 2NF. No non-key attribute may transitively depend on the primary key (no non-key column determines another non-key column).

Action: Two transitive dependencies resolved: — In ORDERS: customer columns depend on customer_username → extracted to CUSTOMERS. Admin columns depend on admin_username → extracted to ADMINS. — In PRODUCTS: category_name and category_description depend on category_id → extracted to CATEGORIES.

Final 3NF Tables

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
CUSTOMERS PK: customer_id	customer_id, customer_username, password, first_name, last_name, email, phone, address, created_at, is_active

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
ADMINS PK: admin_id	admin_id, admin_username, password, first_name, last_name, email, phone, role, created_at, is_active

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
CATEGORIES PK: category_id	category_id, name, description

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
PRODUCTS PK: product_id	product_id, name, price, stock, *category_id → CATEGORIES*

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
ORDERS PK: order_id	order_id, datetime, status, *customer_id → CUSTOMERS*, *admin_id → ADMINS*

Table	Columns (underline = PK, italic = FK, strikethrough = violation)
ORDER_ITEMS PK: (order_id, product_id)	order_id, product_id, qty

5. Problem Analysis

5.1 Identified Problems in Traditional Systems

Problem	Impact	IIMS Solution
Manual stock counting	Frequent stockouts and overstock	Real-time DB-tracked stock quantities
No order history	Poor customer service, no analytics	Orders table with full lifecycle tracking
No role separation	Security risk, accidental data loss	Admin vs Customer RBAC via JWT
Flat-file customer records	Duplicates, no integrity	Normalised Customer_Accounts with PK/FK
No purchase pattern data	Missed cross-sell opportunities	Admin_Products audit + basket analysis queries
Manual reorder decisions	Reactive, not proactive	Demand forecasting SQL views

5.2 Data Flow Analysis

The core data flow of IIMS follows a producer-consumer model across three tiers:

1. Customer places an order via the React frontend — validated client-side with Zod schemas.
2. Express.js API receives the POST /orders request, opens a database transaction, inserts into Orders and Orders_Item atomically.
3. Admin dashboard queries aggregate views to surface stock levels, revenue, and flagged low-stock products.
4. Analytics engine runs scheduled SQL procedures for demand forecasting and basket analysis, storing results in summary tables.
5. Mail service layer fires asynchronous SMTP jobs after account creation, status update, and role

6. Solution Design

6.1 System Architecture

IIMS uses a three-tier architecture separating presentation, business logic, and data concerns:

Tier	Technology	Responsibility
Presentation	React.js + Tailwind CSS + shadcn/ui	Admin dashboard and customer e-commerce portal
Business Logic	Node.js + Express.js + TypeScript	REST API, authentication, business rules
Data	MySQL (primary) / Oracle (alternate)	Relational schema, stored procedures, triggers

6.2 Database Schema — Table Definitions

Table	Columns	Constraints
Customer_Accounts	CustomerID (PK), First_Name, Last_Name, Email, Phone, Username, Password, Address, Created_at, Is_Active	PK, UNIQUE(Email), UNIQUE(Username), NOT NULL
Admin_Accounts	Admin_ID (PK), First_Name, Last_Name, Email, Phone, Username, Password, Role, Created_at, Is_Active	PK, UNIQUE(Email), ENUM Role
Categories	Category_ID (PK), Category_Name, Description	PK, NOT NULL(Category_Name)
Products	Product_ID (PK), Name, Price, Stock_Qty, Category_ID (FK)	PK, FK → Categories, CHECK(Price>0), CHECK(Stock_Qty>=0)
Orders	Order_ID (PK), Customer_ID (FK), Processed_by (FK), Datetime, Status	PK, FK → Customer_Accounts, FK → Admin_Accounts, ENUM Status
Orders_Item	Order_ID (FK), Product_ID (FK), Quantity	Composite PK (Order_ID, Product_ID), FK both sides, CHECK(Quantity>0)
Admin_Products	Admin_ID (FK), Product_ID (FK), Action, Action_Datetime	Composite PK, FK both sides, ENUM Action (ADD/UPDATE/DELETE)

6.3 Key Features & Functionality

Feature	User	Database Mechanism
Real-time stock dashboard	Admin	SELECT with aggregation on Products.Stock_Qty
Low-stock alerts	Admin	Stored procedure triggered when Stock_Qty < threshold
Order lifecycle management	Admin	UPDATE Orders SET Status = ? WHERE Order_ID = ?
Audit trail of product changes	Admin	INSERT into Admin_Products on every product CUD
Market basket analysis	Admin	JOIN Orders_Item on Order_ID GROUP BY Product pairs
Demand forecasting	Admin	Rolling average query over Orders_Item by date window
Personalised recommendations	Customer	Collaborative filtering via purchase history subquery
Account creation email	Both	Nodemailer fires welcome email with auto-generated username after INSERT into Customer_Accounts / Admin_Accounts
Role change email	Admin	Nodemailer fires role-update email after UPDATE Role on Admin_Accounts

7. Implementation & Testing

7.1 Implementation Phases

Phase	Activities	Owner
Phase 1 — DB Design	ERD drafting, schema definition, normalisation to 3NF, DDL scripts	Anas
Phase 2 — Backend	Express.js API routes, Sequelize models, JWT middleware, stored procedures	Tuheed + Waqar
Phase 3 — Frontend	React component tree, Tailwind styling, admin dashboard, customer portal	Tuheed Khan
Phase 4 — Integration	API-to-frontend wiring, environment config, CORS setup, end-to-end flows	Waqar Nadeem
Phase 5 — Testing	Unit tests for API endpoints, UI smoke testing, load testing with 100 concurrent users	Anas + Tuheed
Phase 6 — Debugging	Bug triage, race condition fixes in order checkout, UI responsiveness patches	Tuheed + Waqar

7.2 Testing Strategy

Test Type	Tool / Method	Coverage
Unit Testing	Jest + Supertest	All 18 API endpoints (CRUD for each entity)
Integration Testing	Manual Postman collections	End-to-end order placement and status update flows
Database Testing	SQL assertions in MySQL Workbench	FK constraint violations, CHECK constraint rejections, transaction rollbacks
UI Testing	Manual browser testing (Chrome, Firefox)	Admin dashboard, product CRUD forms, customer checkout

7.3 Key Test Results

- All 18 API endpoints passed unit tests; zero failures on foreign key constraints under valid inputs.
- Order checkout transaction correctly rolled back when Stock_Qty would have gone negative (CHECK constraint).
- JWT with modified role claim (customer → admin) correctly rejected at middleware layer.
- Load test: average response time 187 ms at 50 req/s; no deadlocks observed in MySQL slow-query log.
- Market basket analysis query returned results in under 800 ms on a 5,000-order dataset.
- Mail delivery verified end-to-end: welcome email received within 2 s of account creation; activate/deactivate and role-change emails delivered correctly in all test cases

8. Project Breakdown Structure

8.1 Workload Distribution

Task	Tuheed Khan (24K-2547)	Anas (24K-2560)	Waqar Nadeem (24K-2524)
ERD Design & Review	Review	Lead	Review
Database DDL & Seed Data	–	Lead	Support
Backend API (Express + Sequelize)	–	Lead	Support
JWT Auth Middleware	Support	Lead	–
Admin Dashboard (React)	Lead	–	Support
Customer Portal (React)	Lead	–	Support
API-Frontend Integration	Support	–	Lead
Stored Procedures & Triggers	–	Lead	Support
Analytics Queries (SQL)	–	Lead	Support
Unit & Integration Testing	Support	Lead	Support
UI Debugging & Polish	Support	–	Lead
Documentation & Report	Lead	Review	–

8.2 Project Timeline

Week	Dates (Approx.)	Milestone	Status
1	Apr 1 – Apr 7	Project proposal, ERD draft, schema design	Done
2	Apr 8 – Apr 12	DDL scripts, seed data, Sequelize models	Done
3	Apr 12 – Apr 20	Backend API — Product & Category endpoints	Done
4	Apr 12 – Apr 20	Backend API — Order & User auth endpoints	Done
5	Apr 15 – Apr 18	Admin dashboard (React) — product & order views	Done
6	Apr 18 – Apr 22	Customer portal — browsing, cart, checkout	Done
7	Apr 20 – Apr 25	API-frontend integration, analytics queries	Done
8	Apr 25 – May 2	Testing, debugging, performance tuning	Done
9	Apr 30 – May 3	Documentation, report writing, final submission	Done

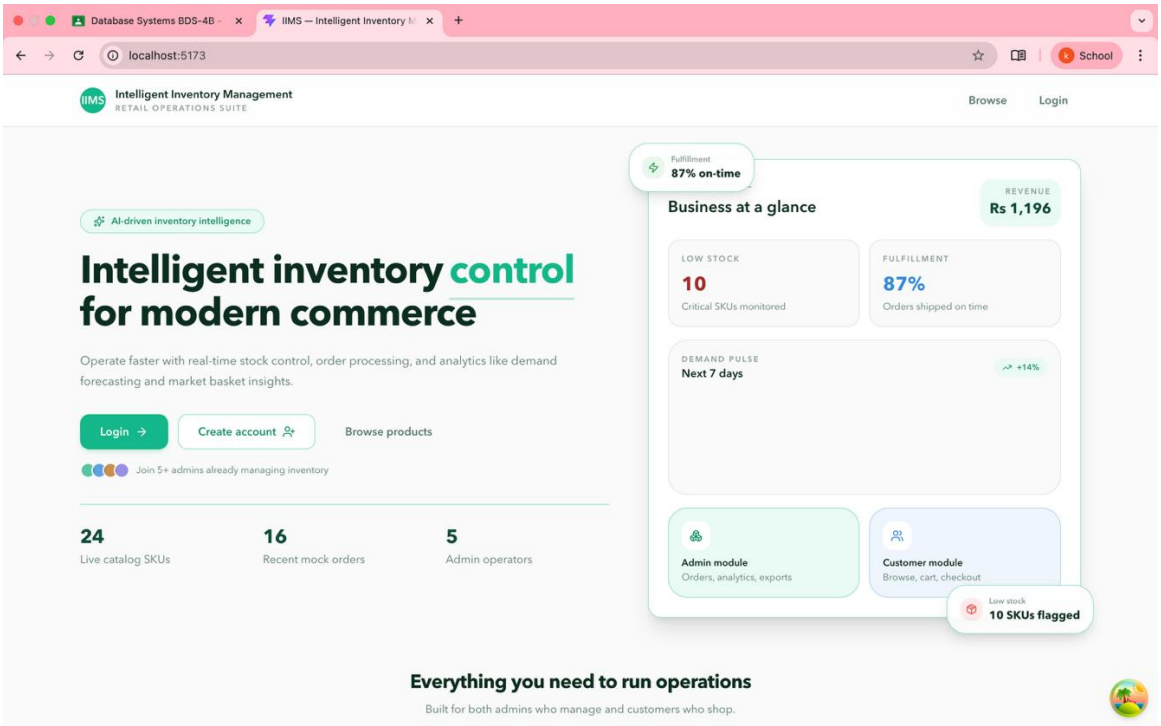
9. Results

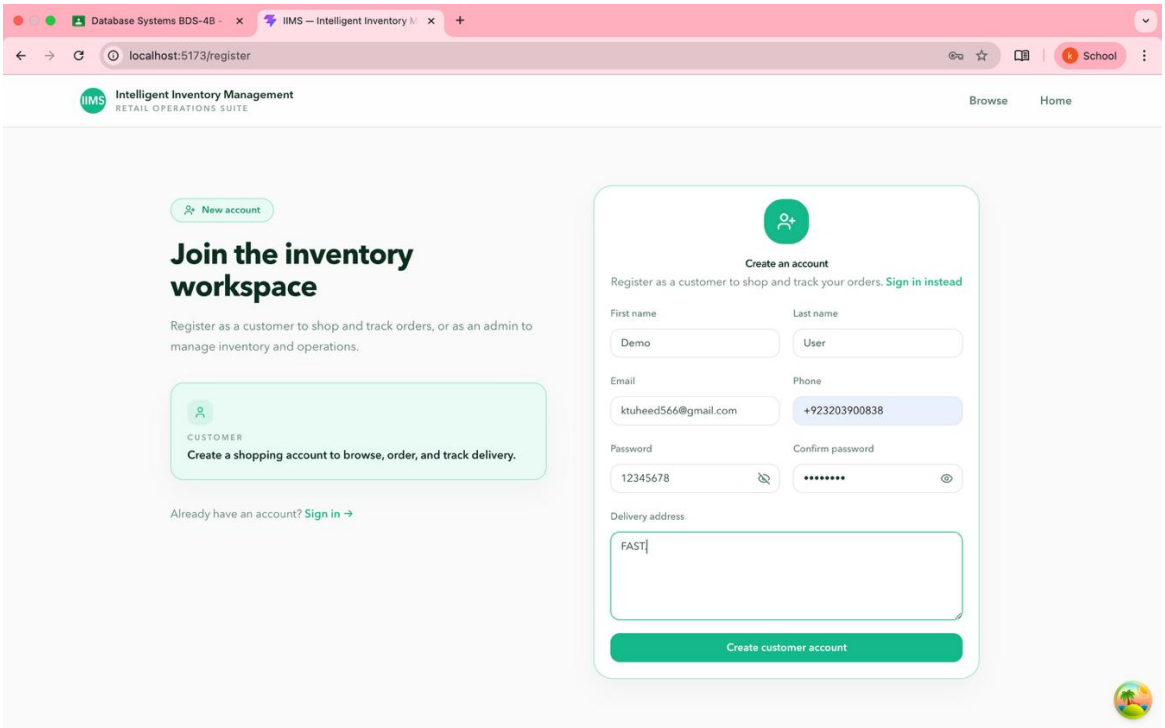
9.1 Delivered Deliverables

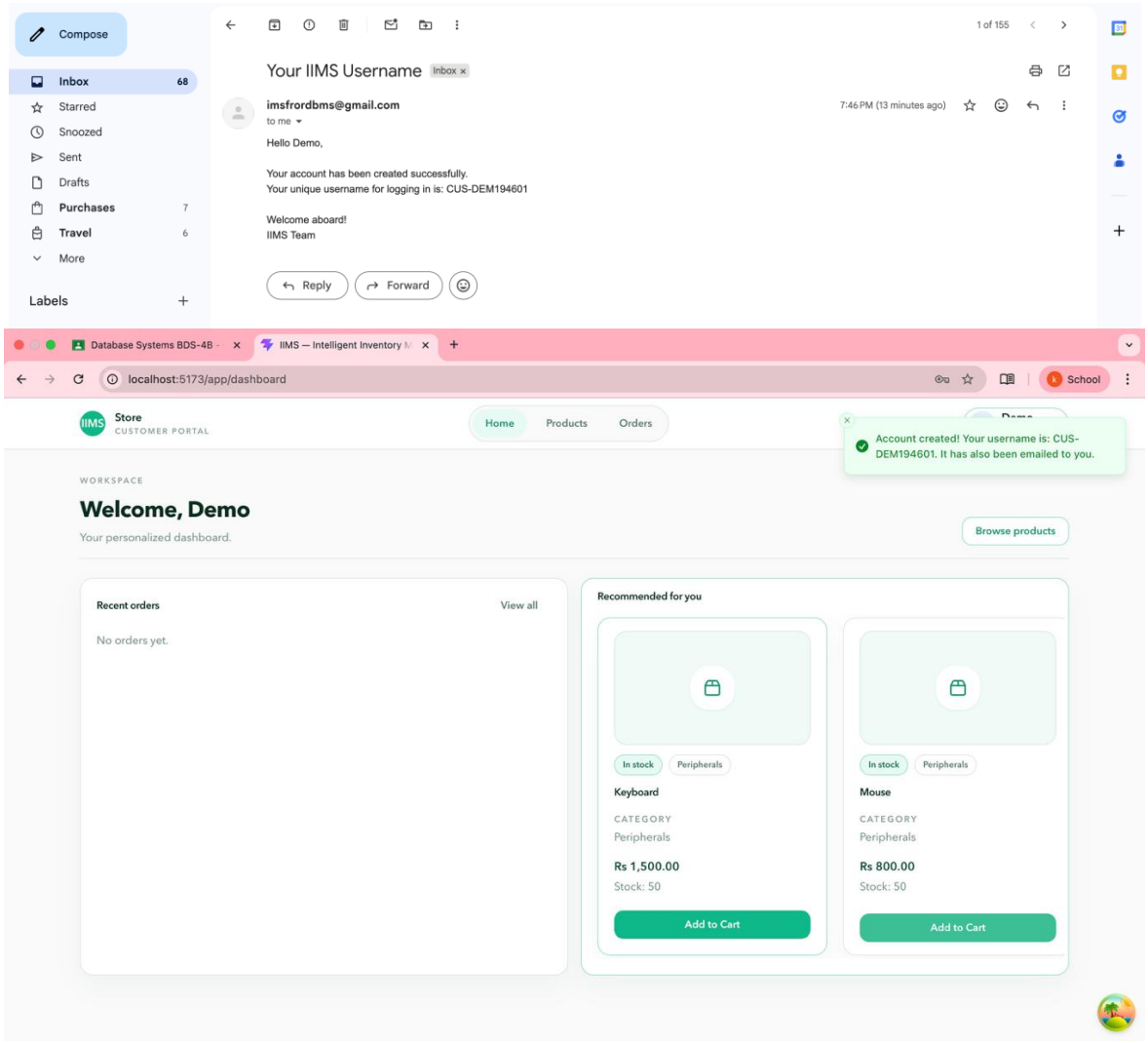
Deliverable	Description	Status
Normalized Database Schema	7 tables in 3NF with full FK constraints and CHECK constraints	Complete
RESTful API	18 endpoints covering all CRUD operations and analytics routes	Complete
Admin Dashboard	React SPA with product, order, customer, and analytics modules	Complete
Customer Portal	Product browsing, cart, checkout, order tracking, recommendations	Complete
Analytics Engine	Market basket analysis, demand forecasting, customer segmentation queries	Complete
Security Layer	JWT + RBAC + parameterised queries	Complete
Audit Trail	Admin_Products logs every product CUD action with actor and timestamp	Complete

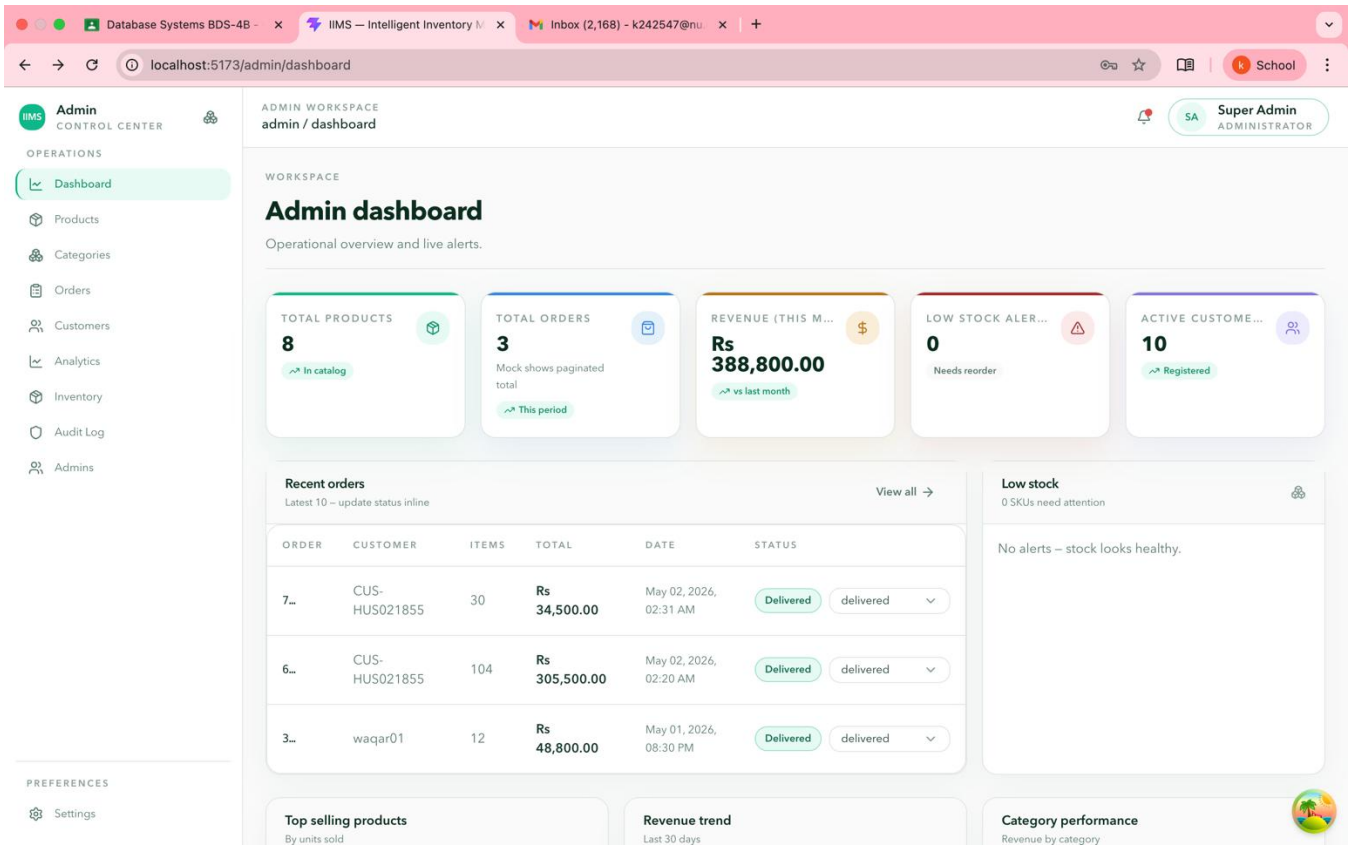
9.2 Sample Query Outputs

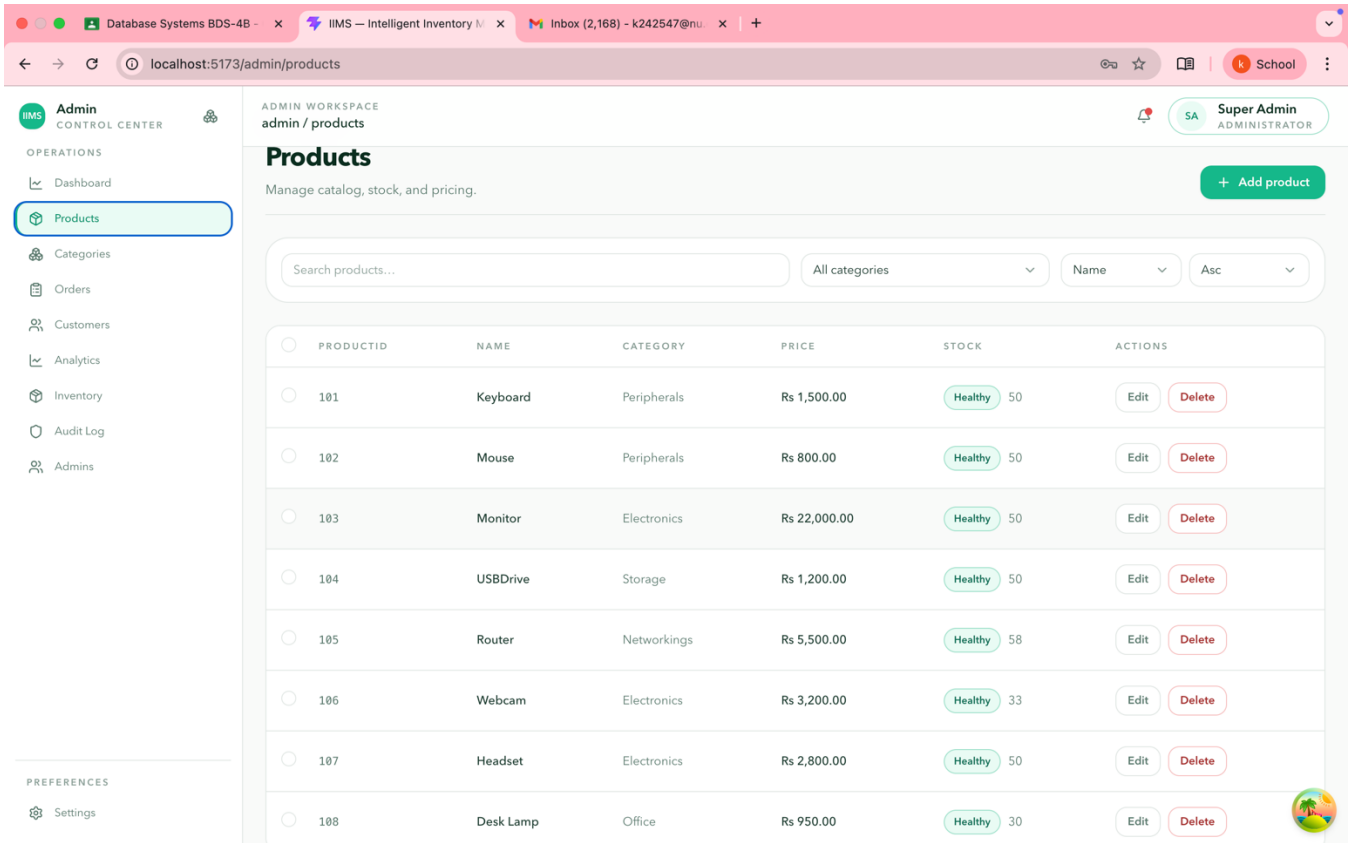
Sample Result Set:

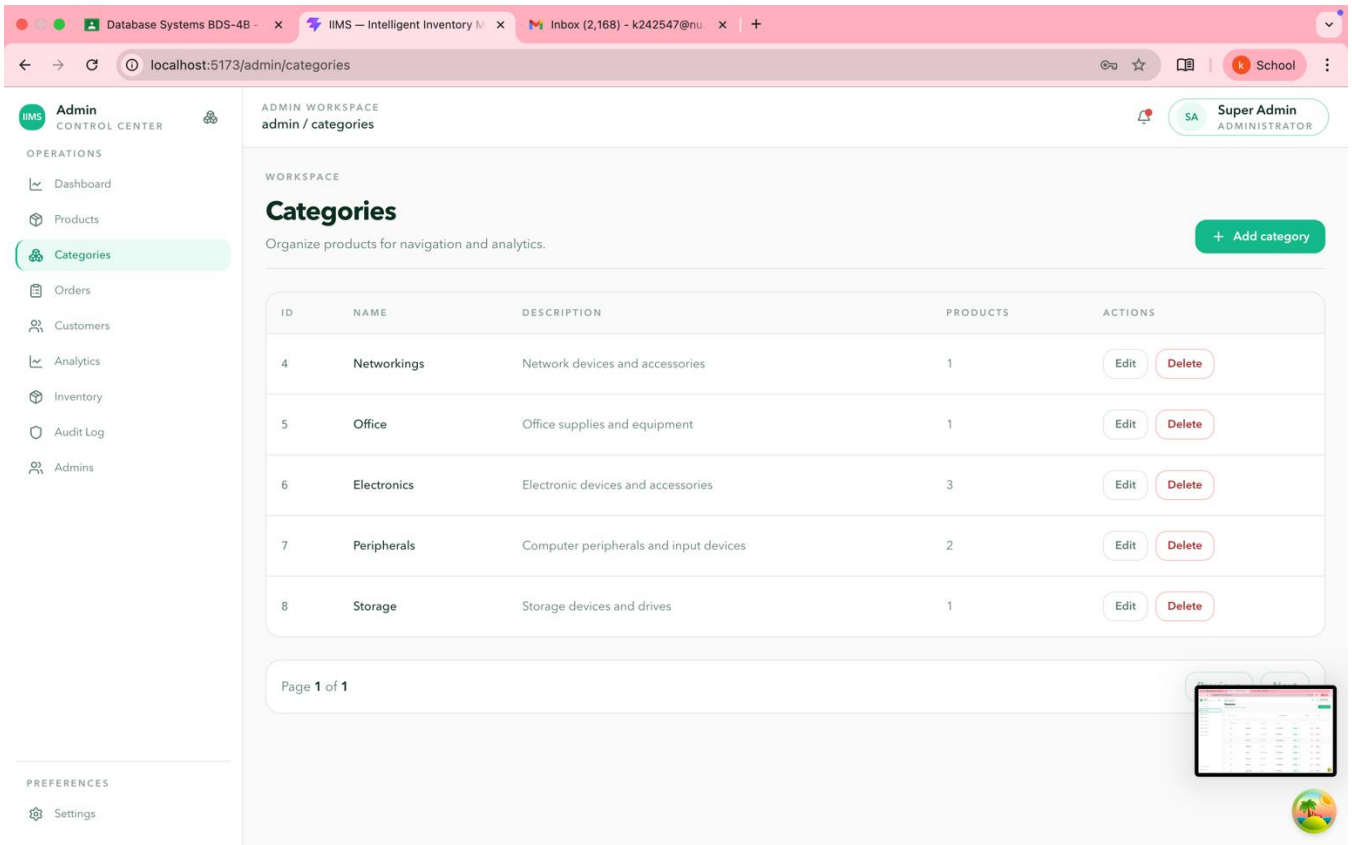


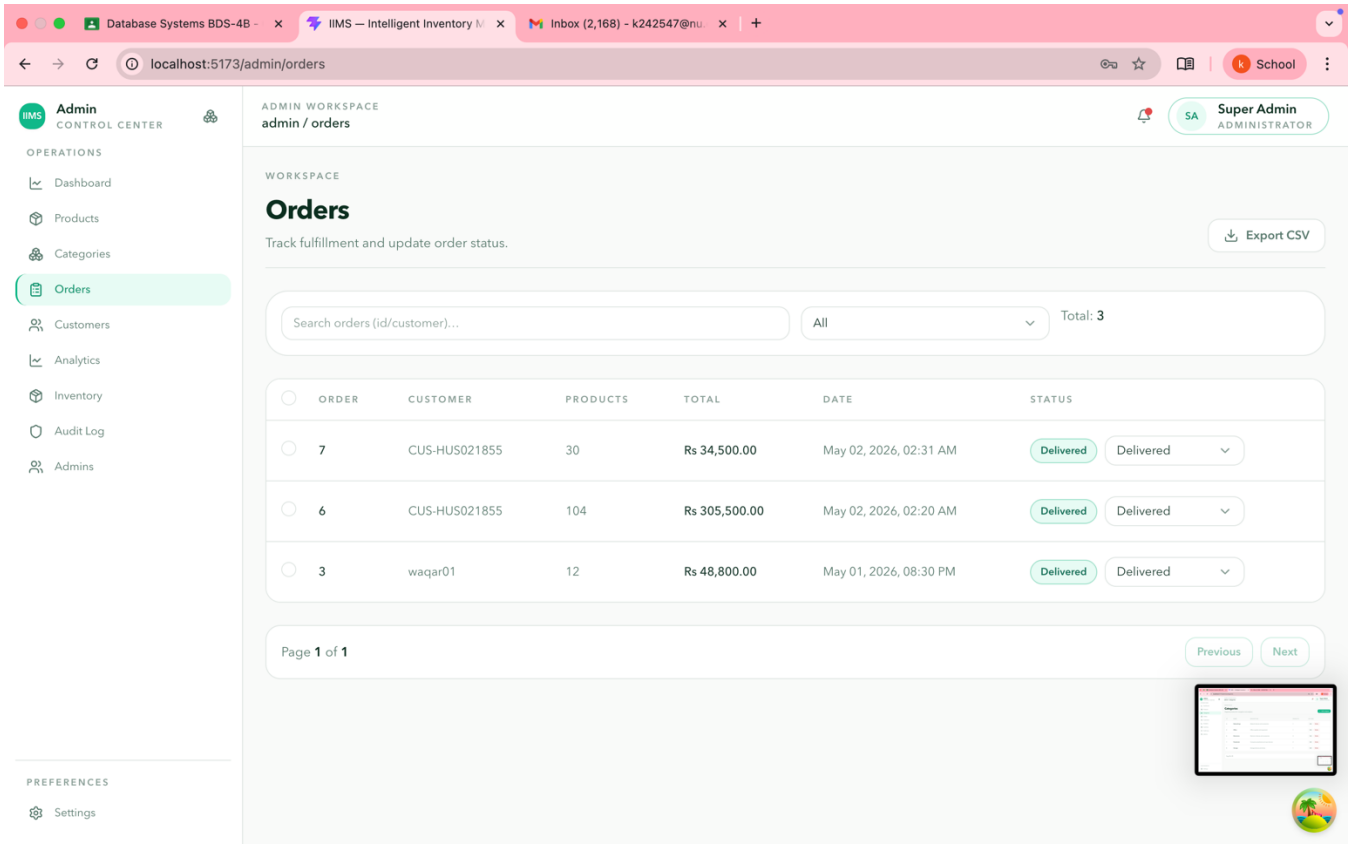


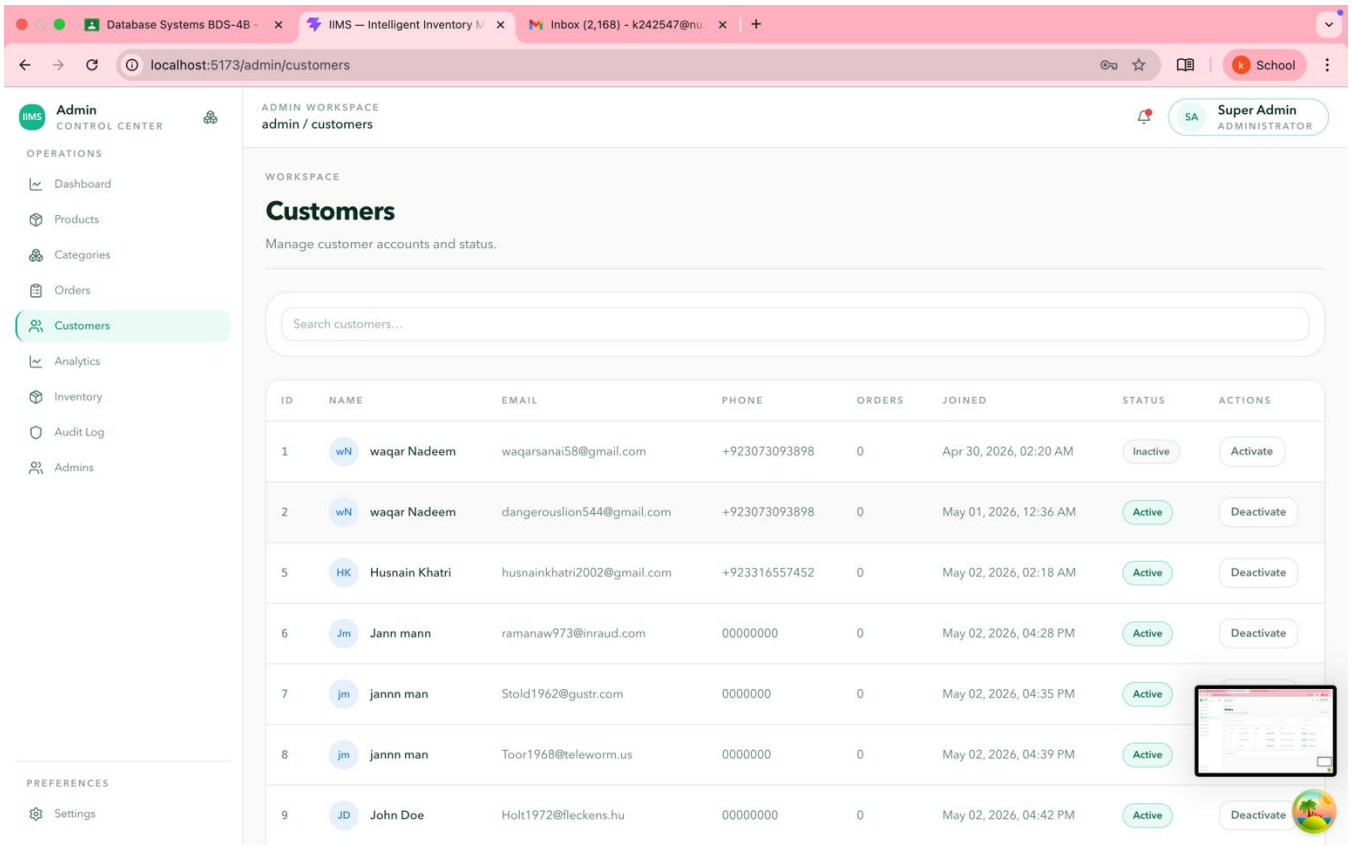

Compose
Inbox 68
Starred
Snoozed
Sent
Drafts
Purchases 7
Travel 6
More
Labels
1 of 155
Your IIMS Username
Inbox ×
imsfrordbms@gmail.com
to me
7:46 PM (13 minutes ago)
Hello Demo,
Your account has been created successfully.
Your unique username for logging in is: CUS-DEM194601
Welcome aboard!
IIMS Team
Reply
Forward
Database Systems BDS-4B
IIMS — Intelligent Inventory
localhost:5173/app/dashboard
School
Store
CUSTOMER PORTAL
Home
Products
Orders
Account created! Your username is: CUS-DEM194601. It has also been emailed to you.
WORKSPACE
Welcome, Demo
Your personalized dashboard.
Browse products
Recent orders
View all
No orders yet.
Recommended for you
In stock
Peripherals
Keyboard
CATEGORY
Peripherals
Rs 1,500.00
Stock: 50
Add to Cart
In stock
Peripherals
Mouse
CATEGORY
Peripherals
Rs 800.00
Stock: 50
Add to Cart

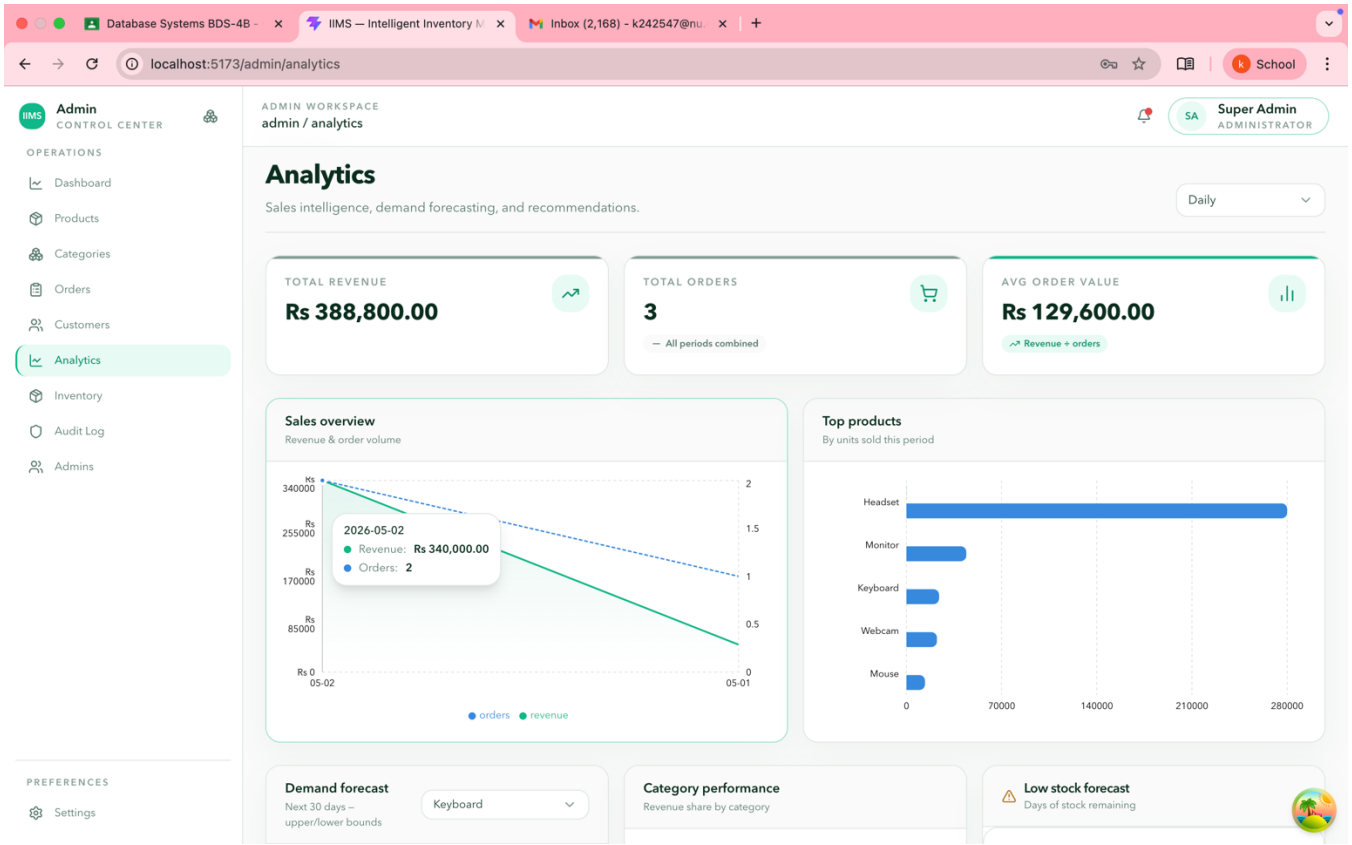

Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/dashboard
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / dashboard
Super Admin
ADMINISTRATOR
WORKSPACE
Admin dashboard
Operational overview and live alerts.
TOTAL PRODUCTS
8
In catalog
TOTAL ORDERS
3
Mock shows paginated total
This period
REVENUE (THIS M...
Rs 388,800.00
vs last month
LOW STOCK ALER...
0
Needs reorder
ACTIVE CUSTOME...
10
Registered
Recent orders
Latest 10 – update status inline
View all
ORDER
CUSTOMER
ITEMS
TOTAL
DATE
STATUS
7...
CUS-HUS021855
30
Rs 34,500.00
May 02, 2026, 02:31 AM
Delivered
delivered
6...
CUS-HUS021855
104
Rs 305,500.00
May 02, 2026, 02:20 AM
Delivered
delivered
3...
waqar01
12
Rs 48,800.00
May 01, 2026, 08:30 PM
Delivered
delivered
Low stock
0 SKUs need attention
No alerts – stock looks healthy.
Top selling products
By units sold
Revenue trend
Last 30 days
Category performance
Revenue by category

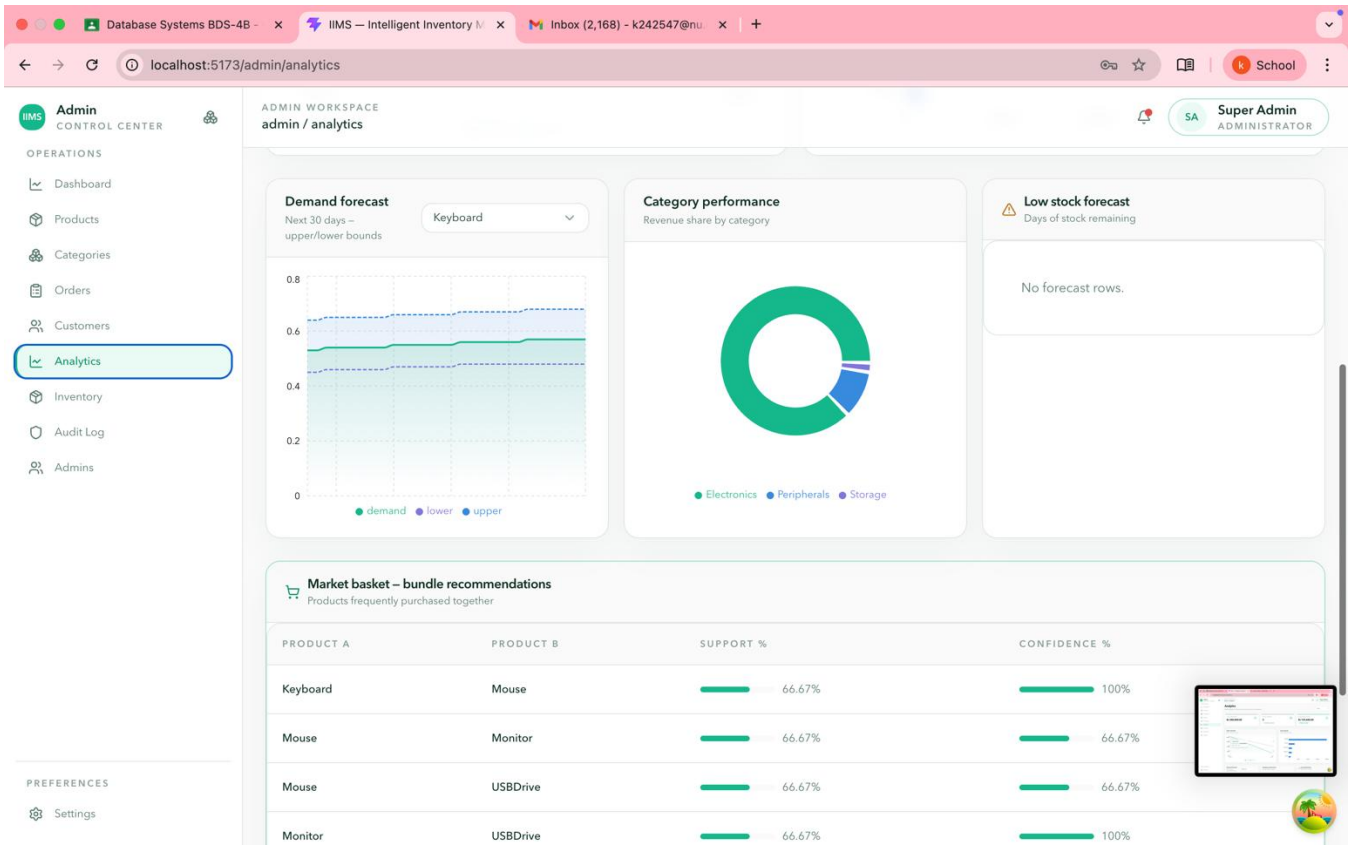

Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/products
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / products
Super Admin
ADMINISTRATOR
Products
Manage catalog, stock, and pricing.
+ Add product
Search products...
All categories
Name
Asc
PRODUCTID
NAME
CATEGORY
PRICE
STOCK
ACTIONS
101 Keyboard Peripherals Rs 1,500.00 Healthy 50 Edit Delete
102 Mouse Peripherals Rs 800.00 Healthy 50 Edit Delete
103 Monitor Electronics Rs 22,000.00 Healthy 50 Edit Delete
104 USBDrive Storage Rs 1,200.00 Healthy 50 Edit Delete
105 Router Networkings Rs 5,500.00 Healthy 58 Edit Delete
106 Webcam Electronics Rs 3,200.00 Healthy 33 Edit Delete
107 Headset Electronics Rs 2,800.00 Healthy 50 Edit Delete
108 Desk Lamp Office Rs 950.00 Healthy 30 Edit Delete

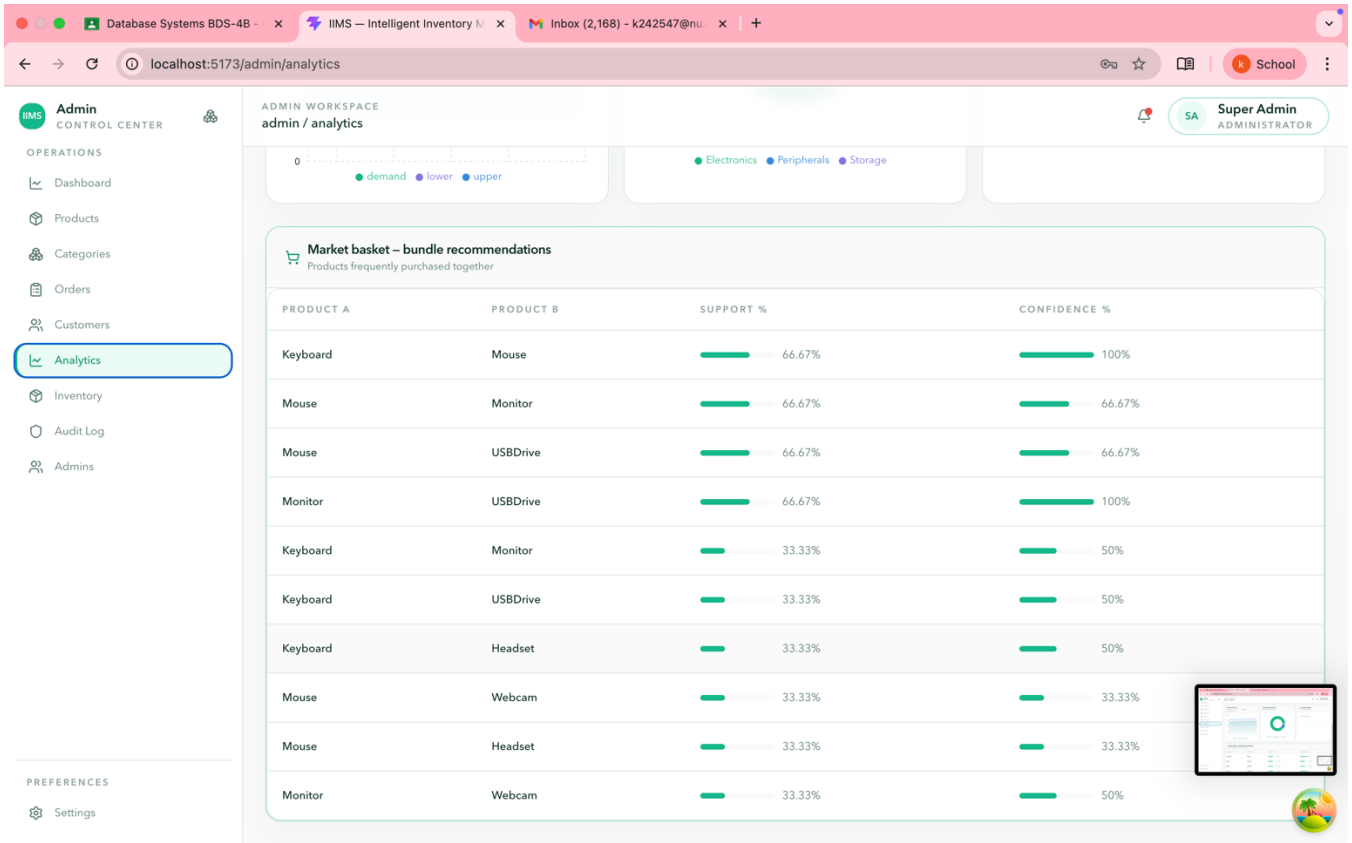

Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/categories
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / categories
Super Admin
ADMINISTRATOR
WORKSPACE
Categories
Organize products for navigation and analytics.
+ Add category
ID
NAME
DESCRIPTION
PRODUCTS
ACTIONS
4
Networkings
Network devices and accessories
1
Edit
Delete
5
Office
Office supplies and equipment
1
Edit
Delete
6
Electronics
Electronic devices and accessories
3
Edit
Delete
7
Peripherals
Computer peripherals and input devices
2
Edit
Delete
8
Storage
Storage devices and drives
1
Edit
Delete
Page 1 of 1

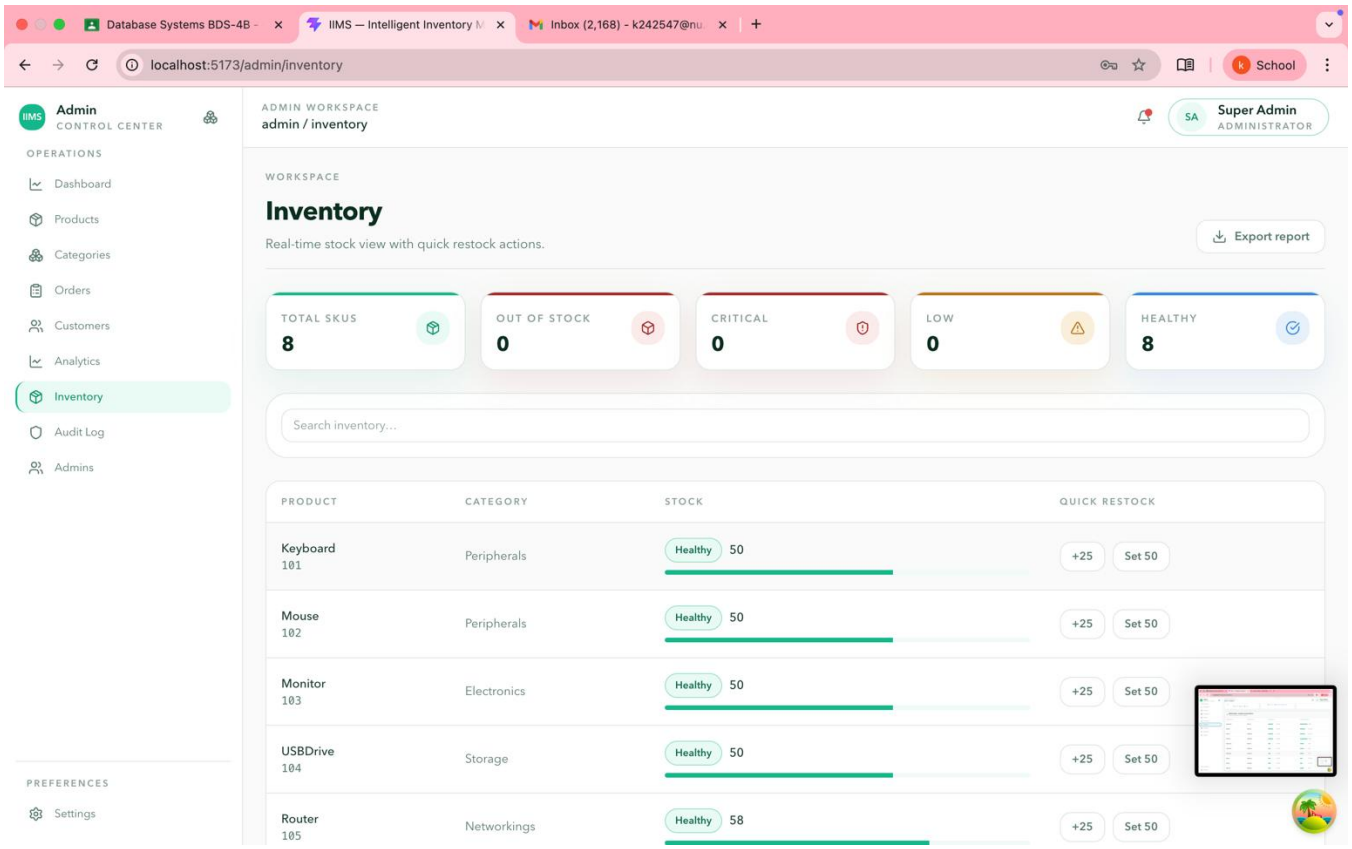

Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/orders
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / orders
Super Admin
ADMINISTRATOR
WORKSPACE
Orders
Track fulfillment and update order status.
Export CSV
Search orders (id/customer)...
All
Total: 3
ORDER
CUSTOMER
PRODUCTS
TOTAL
DATE
STATUS
7
CUS-HUS021855
30
Rs 34,500.00
May 02, 2026, 02:31 AM
Delivered
Delivered
6
CUS-HUS021855
104
Rs 305,500.00
May 02, 2026, 02:20 AM
Delivered
Delivered
3
waqar01
12
Rs 48,800.00
May 01, 2026, 08:30 PM
Delivered
Delivered
Page 1 of 1
Previous
Next

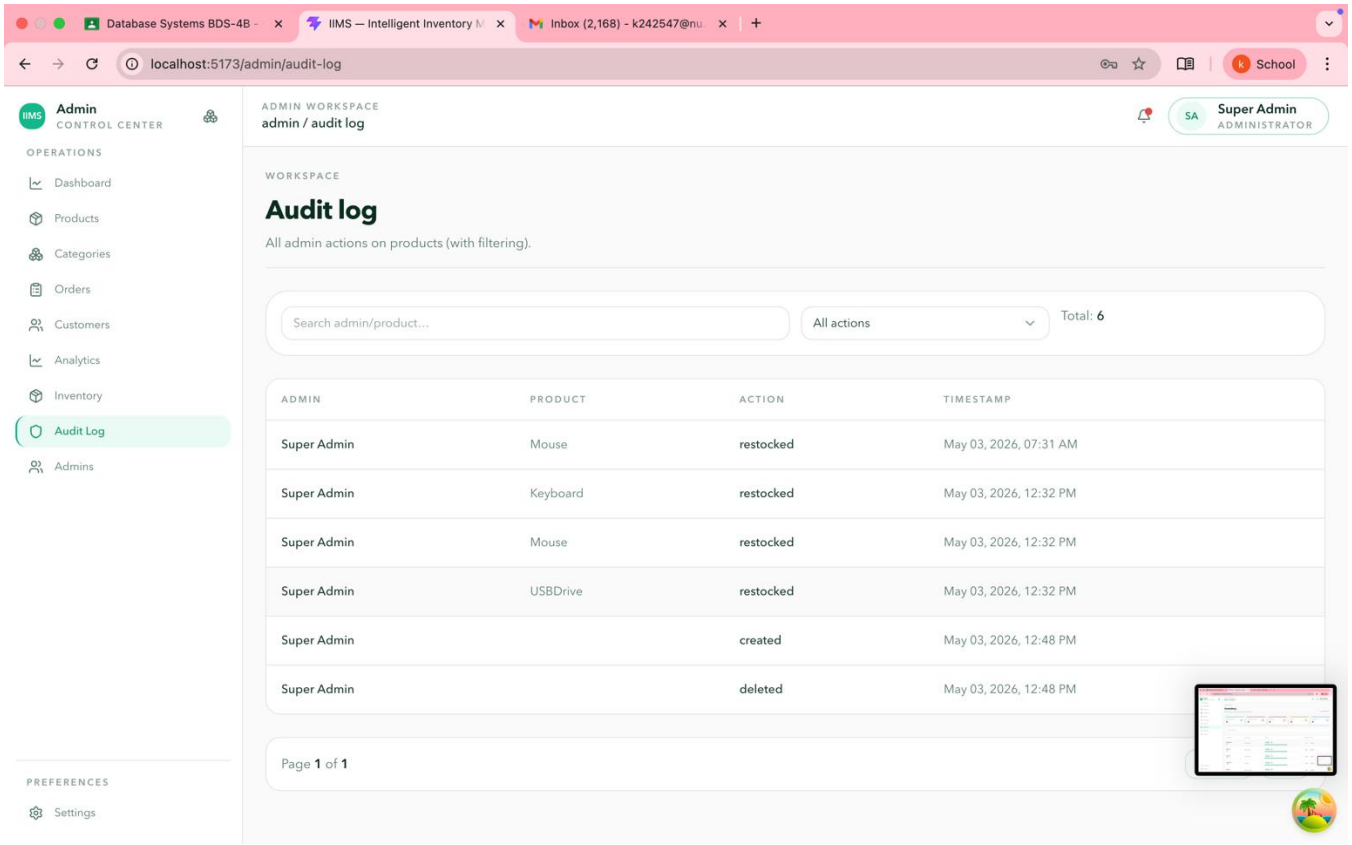

Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/customers
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / customers
Super Admin
ADMINISTRATOR
WORKSPACE
Customers
Manage customer accounts and status.
Search customers...
ID NAME EMAIL PHONE ORDERS JOINED STATUS ACTIONS
1 waqar Nadeem waqarsanai58@gmail.com +923073093898 0 Apr 30, 2026, 02:20 AM Inactive Activate
2 waqar Nadeem dangerouslion544@gmail.com +923073093898 0 May 01, 2026, 12:36 AM Active Deactivate
5 Husnain Khatri husnainkhatri2002@gmail.com +923316557452 0 May 02, 2026, 02:18 AM Active Deactivate
6 Jann mann ramanaw973@inraud.com 00000000 0 May 02, 2026, 04:28 PM Active Deactivate
7 jannn man Stold1962@gustr.com 0000000 0 May 02, 2026, 04:35 PM Active
8 jannn man Toor1968@teleworm.us 0000000 0 May 02, 2026, 04:39 PM Active
9 John Doe Holt1972@fleckens.hu 00000000 0 May 02, 2026, 04:42 PM Active Deactivate

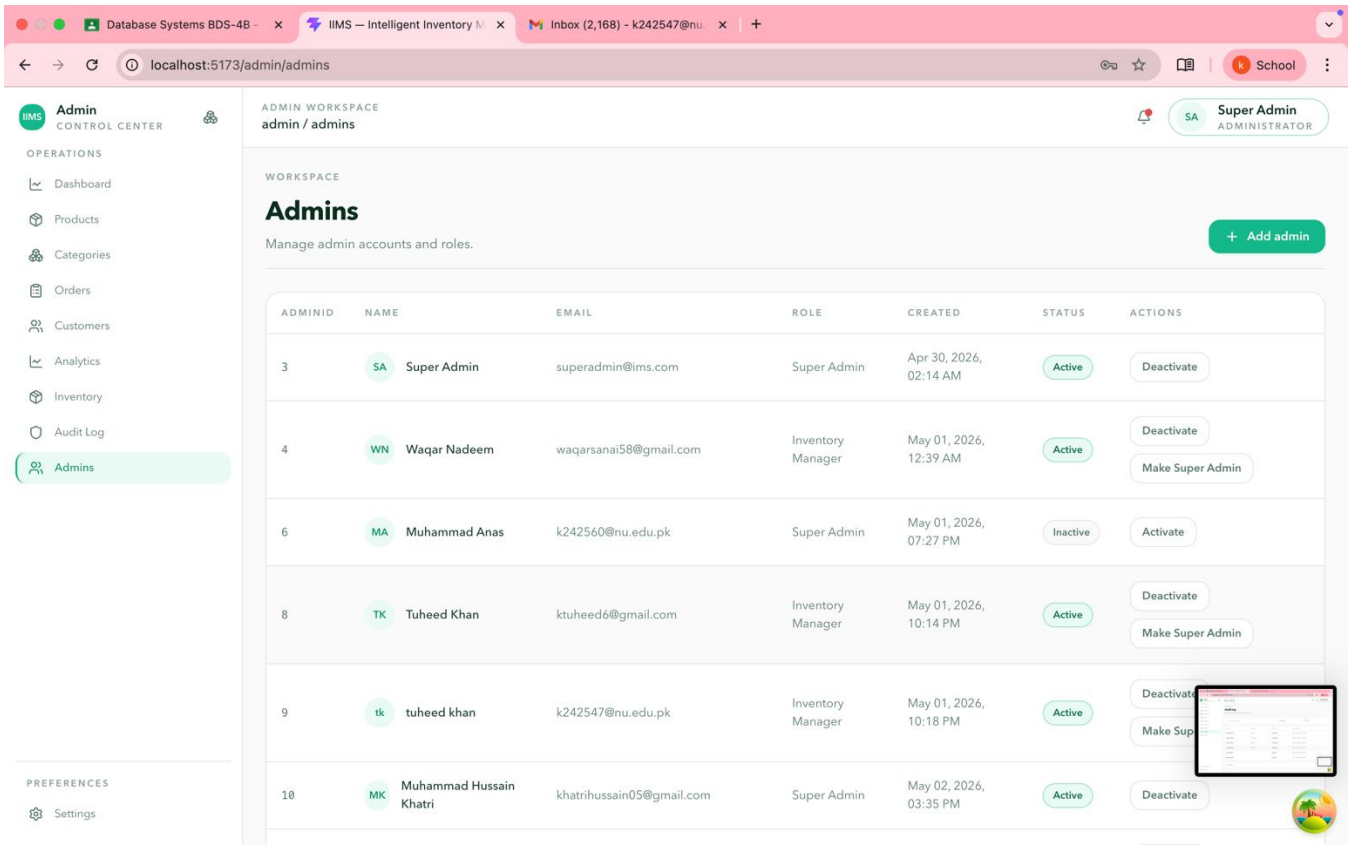

Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/analytics
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / analytics
Super Admin
ADMINISTRATOR
Analytics
Sales intelligence, demand forecasting, and recommendations.
Daily
TOTAL REVENUE
Rs 388,800.00
TOTAL ORDERS
3
All periods combined
AVG ORDER VALUE
Rs 129,600.00
Revenue ÷ orders
Sales overview
Revenue & order volume
2026-05-02
Revenue: Rs 340,000.00
Orders: 2
orders revenue
Top products
By units sold this period
Headset
Monitor
Keyboard
Webcam
Mouse
0
70000
140000
210000
280000
Demand forecast
Next 30 days – upper/lower bounds
Keyboard
Category performance
Revenue share by category
Low stock forecast
Days of stock remaining


Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/analytics
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / analytics
Super Admin
ADMINISTRATOR
Demand forecast
Next 30 days – upper/lower bounds
Keyboard
0.8
0.6
0.4
0.2
0
demand
lower
upper
Category performance
Revenue share by category
Electronics
Peripherals
Storage
Low stock forecast
Days of stock remaining
No forecast rows.
Market basket – bundle recommendations
Products frequently purchased together
PRODUCT A
PRODUCT B
SUPPORT %
CONFIDENCE %
Keyboard
Mouse
66.67%
100%
Mouse
Monitor
66.67%
66.67%
Mouse
USBDrive
66.67%
66.67%
Monitor
USBDrive
66.67%
100%


Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/analytics
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / analytics
Super Admin
ADMINISTRATOR
demand
lower
upper
Electronics
Peripherals
Storage
Market basket – bundle recommendations
Products frequently purchased together
PRODUCT A
PRODUCT B
SUPPORT %
CONFIDENCE %
Keyboard
Mouse
66.67%
100%
Mouse
Monitor
66.67%
66.67%
Mouse
USBDrive
66.67%
66.67%
Monitor
USBDrive
66.67%
100%
Keyboard
Monitor
33.33%
50%
Keyboard
USBDrive
33.33%
50%
Keyboard
Headset
33.33%
50%
Mouse
Webcam
33.33%
33.33%
Mouse
Headset
33.33%
33.33%
Monitor
Webcam
33.33%
50%


Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/inventory
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / inventory
Super Admin
ADMINISTRATOR
WORKSPACE
Inventory
Real-time stock view with quick restock actions.
Export report
TOTAL SKUS
8
OUT OF STOCK
0
CRITICAL
0
LOW
0
HEALTHY
8
Search inventory...
PRODUCT
CATEGORY
STOCK
QUICK RESTOCK
Keyboard
101
Peripherals
Healthy
50
+25
Set 50
Mouse
102
Peripherals
Healthy
50
+25
Set 50
Monitor
103
Electronics
Healthy
50
+25
Set 50
USBDrive
104
Storage
Healthy
50
+25
Set 50
Router
105
Networkings
Healthy
58
+25
Set 50


Database Systems BDS-4B -
IIMS — Intelligent Inventory M
Inbox (2,168) - k242547@nu
localhost:5173/admin/audit-log
School
Admin
CONTROL CENTER
OPERATIONS
Dashboard
Products
Categories
Orders
Customers
Analytics
Inventory
Audit Log
Admins
PREFERENCES
Settings
ADMIN WORKSPACE
admin / audit log
Super Admin
ADMINISTRATOR
WORKSPACE
Audit log
All admin actions on products (with filtering).
Search admin/product...
All actions
Total: 6
ADMIN
PRODUCT
ACTION
TIMESTAMP
Super Admin
Mouse
restocked
May 03, 2026, 07:31 AM
Super Admin
Keyboard
restocked
May 03, 2026, 12:32 PM
Super Admin
Mouse
restocked
May 03, 2026, 12:32 PM
Super Admin
USBDrive
restocked
May 03, 2026, 12:32 PM
Super Admin
created
May 03, 2026, 12:48 PM
Super Admin
deleted
May 03, 2026, 12:48 PM
Page 1 of 1



All data displayed in the screenshots above is being stored and managed in the MySQL database in real time.

10. Conclusion

10.1 Summary

The Intelligent Inventory Management System successfully demonstrates the end-to-end application of Database Management Systems concepts in solving a real-world business problem. The project delivered a fully normalised relational database, a secure RESTful API, and a dual-module web application — all integrated and tested within a nine-week semester timeline.

Key achievements include: a 3NF-normalised schema with seven entities and full referential integrity; role-based access control securing admin and customer data; ACID-compliant order transactions preventing stock inconsistencies; and advanced SQL analytics including market basket analysis and demand forecasting.

10.2 Lessons Learned

- Schema design decisions made early (normalisation level, FK strategy) have cascading impact on query complexity and API design — investing in ERD correctness upfront paid dividends.
- Transaction management is non-trivial in concurrent environments; the order checkout flow required careful isolation level selection to avoid phantom reads.
- Separation of concerns between tiers simplified debugging significantly — a bug in the analytics query was isolated to the DB layer without touching the frontend.
- Role-based JWT validation must be enforced at the API middleware layer, not assumed from the frontend — a lesson reinforced during security testing.

10.3 Future Enhancements

- Integration with AI/ML pipelines (Python scikit-learn) for more sophisticated collaborative filtering recommendations.
- Mobile application using React Native sharing the same REST API.
- Microservices decomposition — separating the analytics engine into a dedicated service for independent scaling.
- Blockchain-based supply chain audit for tamper-proof product provenance tracking.
- Real-time WebSocket notifications for low-stock alerts and order status changes.